

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ**

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ «МИСИС» (НИТУ МИСИС)**

ИНСТИТУТ КОМПЬЮТЕРНЫХ НАУК (ИKN)

КАФЕДРА АВТОМАТИЗИРОВАННЫХ СИСТЕМ УПРАВЛЕНИЯ (АСУ)

Отчет по практике

**Тема: «Realtime Chat App — веб-приложение для обмена
сообщениями в реальном времени»**

Студент: Усманов Ильдар Русланович

Группа: БИВТ-24-2

Москва, 2026

Содержание

1. Предметная область.....	3
2. Анализ требований	5
3. Этапы выполнения работы.....	6
4. Структура базы данных.....	8
5. API и схемы	10
6. Выводы.....	12
7. Источники	13

1. Предметная область

Цель проекта

Целью проекта являлась разработка клиент-серверного приложения, обеспечивающего регистрацию и авторизацию пользователей, создание личных и групповых чатов, обмен сообщениями в режиме реального времени, а также хранение и просмотр истории переписки.

Разрабатываемое приложение предназначено для обмена сообщениями между пользователями в режиме реального времени. Основными возможностями системы являются регистрация пользователей, создание личных и групповых чатов, хранение истории переписки, а также синхронизация сообщений между несколькими клиентами. Использование архитектуры клиент—сервер позволяет разделить ответственность между пользовательским интерфейсом, серверной логикой и системой хранения данных, обеспечивая удобство дальнейшего сопровождения и развития проекта.

В рамках распределения обязанностей автором настоящего отчёта были выполнены проектирование структуры базы данных, разработка SQL-миграций, настройка развёртывания базы данных средствами Docker Compose, разработка клиентской части приложения на основе React и TypeScript, доработка отдельных компонентов серверной части, выявленных в ходе интеграционного тестирования, а также подготовка презентационных материалов проекта и демонстрации результатов разработки.

Стек технологий.

React, TypeScript и Vite — разработка клиентской части приложения; PostgreSQL — система управления базой данных; Docker Compose — средство

контейнеризированного развёртывания базы данных; WebSocket — обмен данными в режиме реального времени; JWT (access token, refresh token) — механизм авторизации пользователей. Серверная часть реализована на языке Go и используется клиентским приложением посредством REST API.

Выбранный стек технологий обусловлен требованиями к производительности и поддержке работы в режиме реального времени. Язык Go обеспечивает эффективную обработку большого количества одновременных подключений, PostgreSQL предоставляет надёжные средства хранения данных, а использование WebSocket позволяет минимизировать задержку при доставке сообщений между пользователями.

Архитектура приложения построена по трёхзвенной модели «клиент — сервер — база данных». Клиентская часть обеспечивает взаимодействие пользователя с системой, серверная часть реализует бизнес-логику приложения, а база данных PostgreSQL предназначена для хранения информации о пользователях, чатах, сообщениях и токенах авторизации.

Разрабатываемое приложение построено по клиент-серверной архитектуре. Клиентская часть отвечает за пользовательский интерфейс и взаимодействие с пользователем, серверная часть реализует бизнес-логику, обработку запросов и обмен сообщениями в режиме реального времени, а система управления базой данных PostgreSQL обеспечивает хранение учетных записей пользователей, информации о чатах, сообщениях и токенах авторизации. Взаимодействие клиента с сервером осуществляется посредством REST API и протокола WebSocket, а сервер взаимодействует с базой данных через слой репозитория.

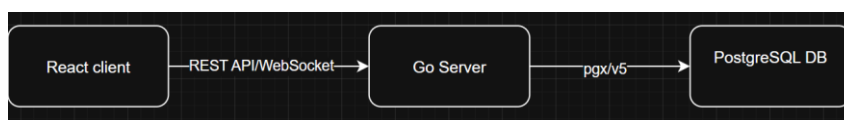


Рисунок 1.1 Архитектура приложения

2. Анализ требований

В ходе анализа требований были определены следующие функциональные задачи серверной части:

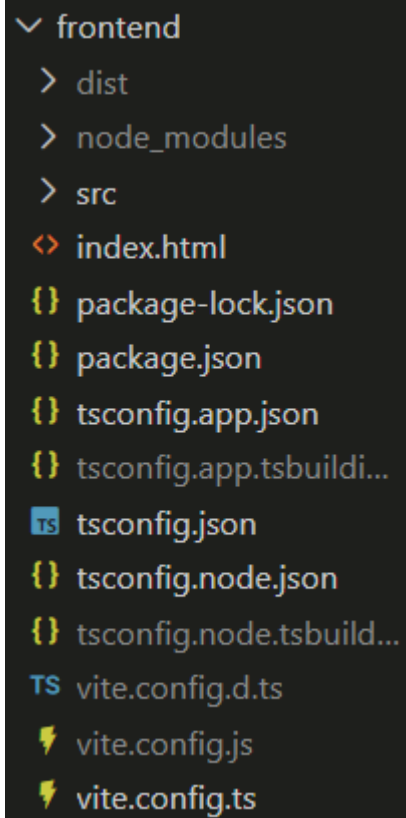
- проектирование структуры реляционной базы данных;
- разработка SQL-миграций для автоматического создания структуры базы данных;
- настройка Docker Compose для развёртывания PostgreSQL;
- разработка клиентской части приложения на React и TypeScript;
- реализация взаимодействия клиента с REST API;
- подключение клиента к WebSocket-серверу;
- отображение списка чатов и истории сообщений;
- реализация поиска сообщений;
- реализация управления участниками групповых чатов;
- проведение интеграционного тестирования и устранение выявленных ошибок.

Особое внимание уделялось согласованию структуры базы данных с серверной частью, корректной обработке запросов к REST API и обеспечению устойчивого обмена сообщениями посредством WebSocket.

3. Этапы выполнения работы

1. Анализ требований и проектирование структуры базы данных.
2. Разработка ER-модели и подготовка SQL-миграций.
3. Настройка Docker Compose для автоматического развёртывания PostgreSQL.
4. Разработка структуры клиентского приложения на базе React, TypeScript и Vite.
5. Реализация страниц регистрации, авторизации, списка чатов и окна переписки.
6. Реализация взаимодействия с REST API и WebSocket.
7. Проведение интеграционного тестирования совместно с серверной частью и устранение выявленных ошибок.

Разработка серверной части выполнялась последовательно — от анализа требований и проектирования архитектуры до реализации отдельных компонентов и их последующей интеграции. Такой подход позволил поэтапно проверить работоспособность каждого модуля и обеспечить корректное взаимодействие всех элементов системы.



```
▼ frontend
  > dist
  > node_modules
  > src
  <> index.html
  {} package-lock.json
  {} package.json
  {} tsconfig.app.json
  {} tsconfig.app.tsbuildi...
  TS tsconfig.json
  {} tsconfig.node.json
  {} tsconfig.node.tsbuild...
  TS vite.config.d.ts
  ⚡ vite.config.js
  ⚡ vite.config.ts
```

The image shows a file explorer view of a 'frontend' directory. It contains subdirectories 'dist', 'node_modules', and 'src'. Files include 'index.html', 'package-lock.json', 'package.json', 'tsconfig.app.json', 'tsconfig.app.tsbuildi...', 'tsconfig.json' (with a TS icon), 'tsconfig.node.json', 'tsconfig.node.tsbuild...', 'vite.config.d.ts' (with a TS icon), 'vite.config.js' (with a lightning bolt icon), and 'vite.config.ts' (with a lightning bolt icon).

Рисунок 3.1 — Структура проекта frontend

4. Структура базы данных

В рамках проекта была спроектирована реляционная база данных, предназначенная для хранения информации о пользователях, чатах, сообщениях и токенах авторизации:

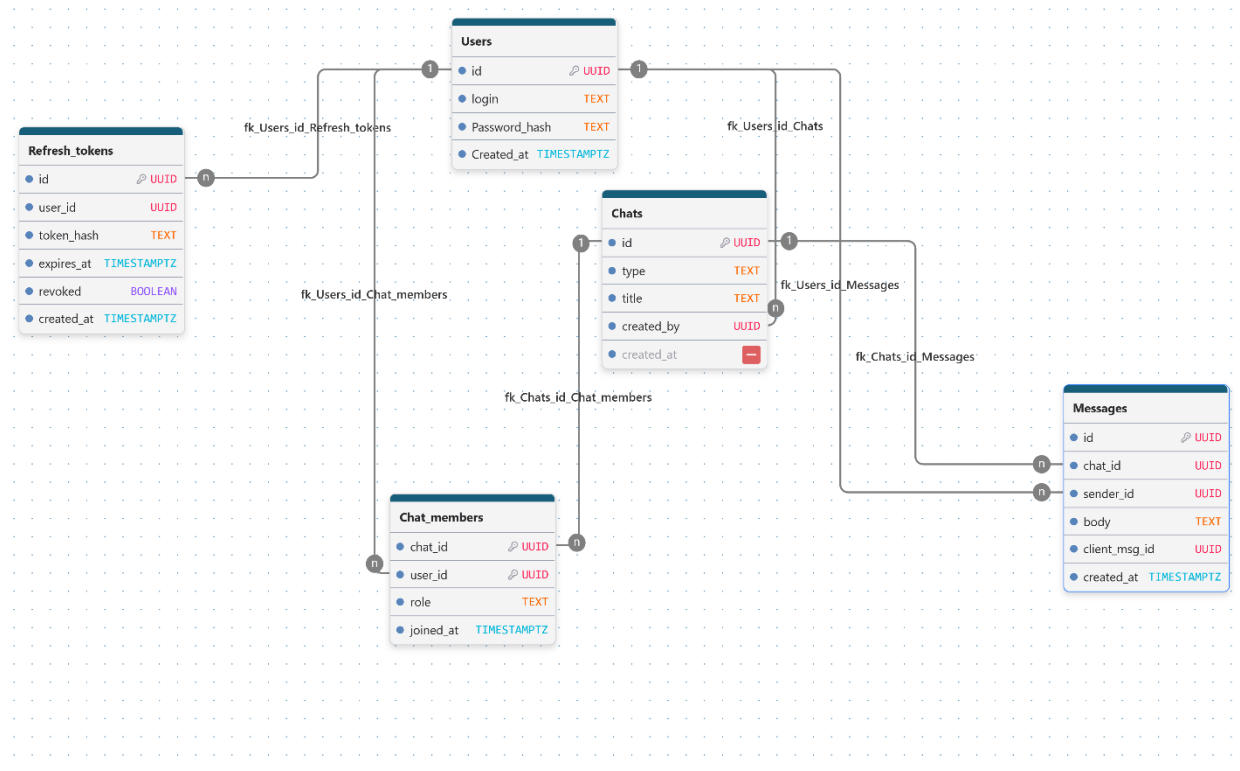


Рисунок 4.1 ER-диаграмма

Таблица users: id, login, password_hash, created_at — учётные записи пользователей.

Таблица refresh_tokens: id, user_id, token_hash, expires_at, created_at — refresh-токены пользователей.

Таблица chats: id, type, title, created_by, created_at — личные и групповые чаты.

Таблица chat_members: chat_id, user_id, role, joined_at — участники чатов и их роли.

Таблица `messages`: `id`, `chat_id`, `sender_id`, `body`, `client_msg_id`, `seq`, `created_at` — сообщения.

Связи между таблицами: один пользователь может являться автором нескольких чатов; один пользователь может являться участником нескольких чатов; один чат может содержать несколько участников и сообщений; каждое сообщение принадлежит одному чату и отправлено одним пользователем; один пользователь может иметь несколько `refresh`-токенов.

Миграции базы данных: `001_users.sql` — создание таблицы пользователей; `002_chats.sql` — создание таблиц чатов и участников; `003_messages.sql` — создание таблицы сообщений; `004_refresh_tokens.sql` — создание таблицы `refresh`-токенов; `005_messages_seq.sql` — добавление порядковой нумерации сообщений; `indexes.sql` — индексы для ускорения запросов.

Создание структуры базы данных осуществляется посредством SQL-миграций, что обеспечивает воспроизводимость процесса развёртывания и возможность контроля изменений схемы базы данных.

Развёртывание базы данных осуществляется средствами `Docker Compose`, обеспечивающими автоматический запуск контейнера `PostgreSQL` и последовательное применение подготовленных SQL-миграций.

5. API и схемы

Клиентская часть приложения взаимодействует с сервером посредством REST API и WebSocket-соединения. Основные используемые конечные точки представлены в таблице.

Таблица 5.1 Эндпоинты REST API

Метод	Эндпоинт	Описание
POST	/api/v1/auth/register	регистрация пользователя
POST	/api/v1/auth/login	авторизация пользователя
POST	/api/v1/auth/refresh	обновление access-токена
POST	/api/v1/auth/logout	завершение сеанса
GET	/api/v1/chats	получение перечня чатов пользователя
POST	/api/v1/chats	создание чата
GET	/api/v1/chats/{id}/members	получение перечня участников чата
POST	/api/v1/chats/{id}/members	добавление участника
DELETE	/api/v1/chats/{id}/members/{userId}	исключение участника
DELETE	/api/v1/chats/{id}	удаление чата либо выход из него
GET	/api/v1/chats/{id}/messages	получение истории сообщений
GET	/api/v1/messages/search	поиск сообщений по тексту

WS	/ws	WebSocket- соединение для обмена сообщениями в реальном времени
----	-----	---

Реализация клиента REST API выполнена с использованием асинхронных HTTP-запросов, а WebSocket-соединение применяется для получения новых сообщений без необходимости периодического опроса сервера.

При успешном выполнении запросов сервер возвращает коды состояния HTTP 200 (OK), 201 (Created) и 204 (No Content). При ошибках используются коды 400 (Bad Request), 401 (Unauthorized), 403 (Forbidden), 404 (Not Found) и 500 (Internal Server Error).

Схема взаимодействия клиентской части с сервером: после успешной авторизации клиент сохраняет access- и refresh-токены и передаёт access-токен в заголовке Authorization: Bearer <token> при обращении к защищённым маршрутам, а также при установлении WebSocket-соединения; при истечении срока действия access-токена клиент автоматически запрашивает новый через /auth/refresh; при выходе из системы токены удаляются на стороне клиента.

Схема отображения ролей на стороне интерфейса: для роли admin отображаются элементы управления добавлением и исключением участников группового чата; для роли member — только функции переписки и выхода из чата. Для личных чатов разграничение ролей не применяется.

6. Выводы

По результатам прохождения практики была спроектирована структура базы данных, разработаны SQL-миграции и выполнена настройка контейнеризированного развёртывания PostgreSQL средствами Docker Compose. Разработана клиентская часть приложения на основе React и TypeScript, реализующая взаимодействие с сервером посредством REST API и WebSocket. В ходе интеграционного тестирования были устранены отдельные ошибки серверной части, выявленные при совместной работе компонентов системы. Полученные результаты позволили закрепить навыки проектирования реляционных баз данных, разработки современных веб-интерфейсов и интеграции клиентского приложения с серверной частью.

7. Источники

1. React Documentation: официальный сайт. – URL: <https://react.dev> (дата обращения: 11.07.2026). – Режим доступа: для зарегистрир. пользователей. – Текст: электронный.
2. TypeScript Documentation: официальный сайт. – URL: typescriptlang.org (дата обращения: 11.07.2026). – Текст: электронный.
3. Vite Documentation: официальный сайт. – URL: <https://vitejs.dev> (дата обращения: 11.07.2026). – Текст: электронный.
4. PostgreSQL Documentation: официальный сайт. – URL: [postgresql.org](https://www.postgresql.org) (дата обращения: 11.07.2026). – Текст: электронный.
5. Docker Compose Documentation: официальный сайт. – URL: [docker.com](https://docs.docker.com/compose/) (дата обращения: 11.07.2026). – Текст: электронный.
6. RFC 6455. The WebSocket Protocol / IETF; edited by I. Fette, A. Melnikov. – 2011. – URL: [ietf.org](https://www.ietf.org/rfc/rfc6455.txt) (дата обращения: 11.07.2026). – Текст: электронный.
7. RFC 7519. JSON Web Token (JWT) / IETF; edited by M. Jones, J. Bradley, N. Sakimura. – 2015. – URL: [ietf.org](https://www.ietf.org/rfc/rfc7519.txt) (дата обращения: 11.07.2026). – Текст: электронный.
8. Репозиторий проекта «realtime-chat-app» [Электронный ресурс]. – URL: <https://github.com/ByakuByaku/realtime-chat-app> (дата обращения: 16.07.2026).